



WELCOME





C++ Basics

Agenda

C++ Bridge Course

C++ Quickstart

Data Types

**Practice
Questions**

How to Learn a New Programming Language?



Analogy

Driving a Car



*Let's say,
You know how to Drive a **Tata Car**.*

Driving a Car

New Car

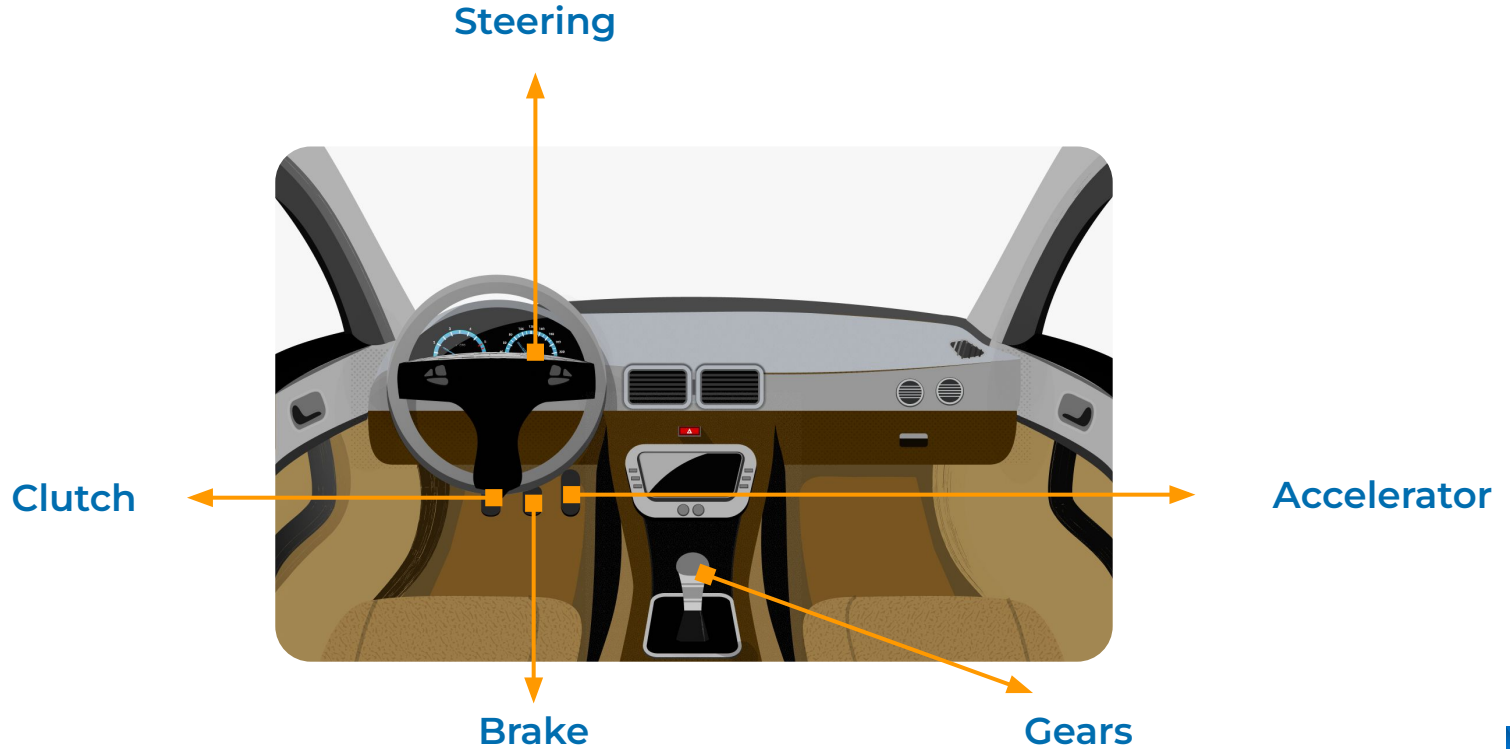
"Now, **You have to Drive a Maruti**

Do you have to learn to drive from scratch?"



Driving a Car

Common Aspects



Driving a Car

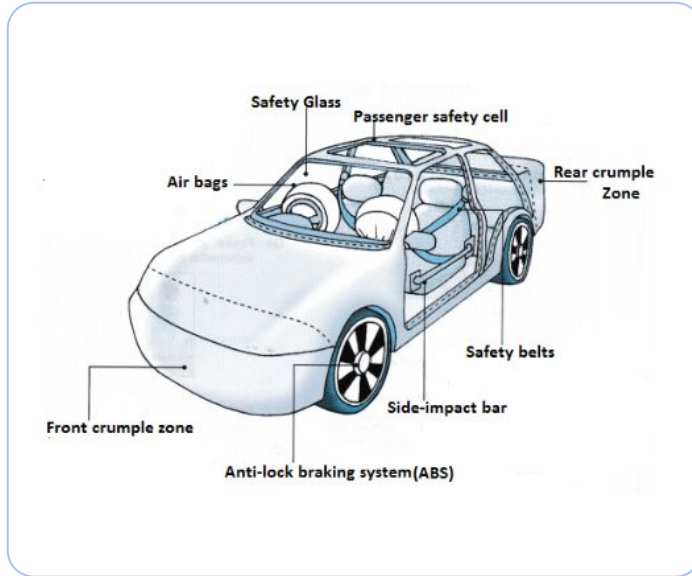
Tata vs Maruti



What are the differences between the two cars?

Driving a Car

Possible Differences



Acceleration / Speed



Seating Capacity



Diesel / Petrol



Engine Power



Wipers



[Source](#)

Driving a Car

New Car

- ? Does driving a new car require a different set of fundamental skills?



Example

Person Applying For a Driver Job

- ❓ Does one need to specify specific cars, like **Tata**, **Honda**, **Mahindra**, etc., when applying for a driving job?



HONDA

mahindra

Driving a Car

Learning a New Car

How much time is needed to learn how to drive different cars?

Tata



Honda



Mahindra



Learning a New Car

Time Required

How much time is needed to learn how to drive different cars?

Tata



~ 30 Days



Honda



~ 2 hours



Mahindra



~ 1 hours

Time Required to Adjust to a New Car

Later, it usually only takes a **few minutes** to **adjust** to a new car.



Comparison with Driving a New Car

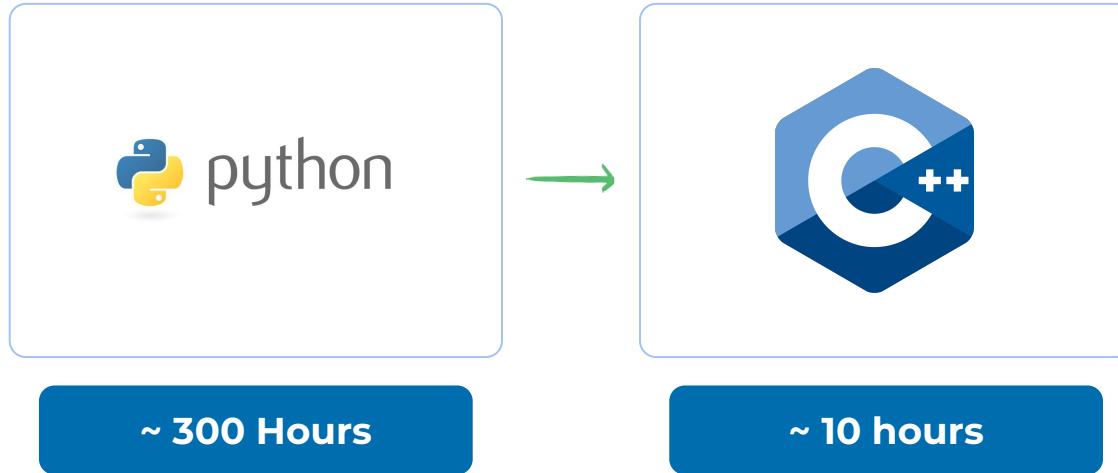
Learning a New Programming Language

- ✓ Learning a **new** programming language is **similar to** learning to drive a new car.
- ✓ A **strong foundation** in **one** programming language can make it **easier** to **learn other languages**.



Learning a New Programming Language

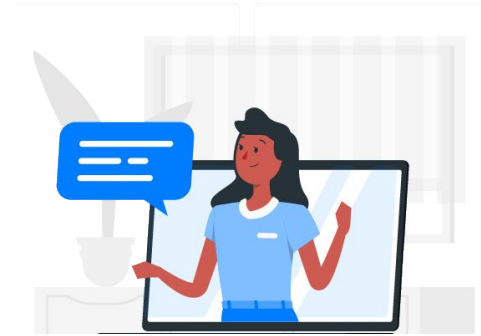
Time Required



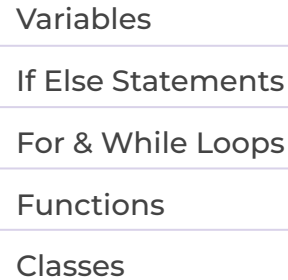
Three key Areas

To become proficient in a programming language, there are three key areas to focus on:

1. Programming Abstractions
2. Problem-Solving Approach
3. Syntax



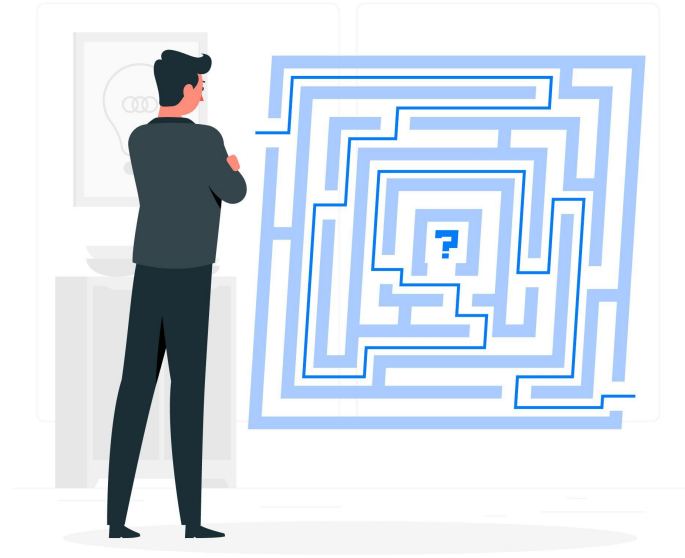
Programming Abstractions



Three key Areas of Focus

Problem-Solving Approach

- ✓ Across all programming languages, the **problem-solving** approach is the **same**.
- ✓ When you **master** it in **one language**, you can **apply** it to **others**.



Three key Areas of Focus

Syntax

- ✓ The **main difference** between programming languages is **Syntax**, which can be learned in just a **few hours**.
- ✓ After that, you need to **practice** more to become proficient in the language.



Why We Chose Python First ?

- ✓ **Python** is the **ideal** language for **learning** programming abstractions, problem-solving approaches, and syntax due to its simplicity.
- ✓ It is a **widely used** programming language that is **popular** across **multiple domains**.



Choosing a new Programming Language

Considerations

Globally, there are around **700** programming languages.



Let's Look at a New Programming Language



New Programming language

C++

- ✓ C++ is a **widely used, object-oriented,** and **generic** programming language.
- ✓ It is utilized by **millions** of **developers** and **powers** a vast **range** of **applications**, from **game engines** to **operating systems**.



C++

Applications of C++



Desktop Applications.



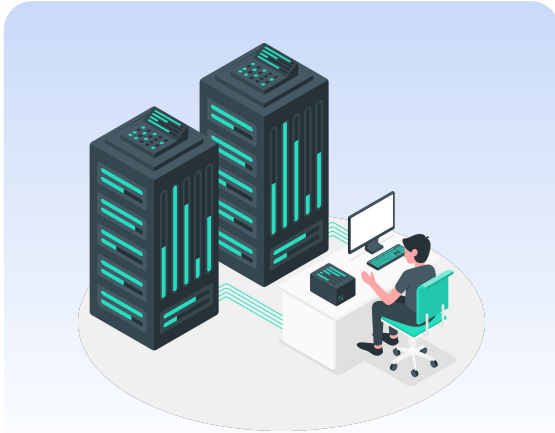
Mobile Applications.



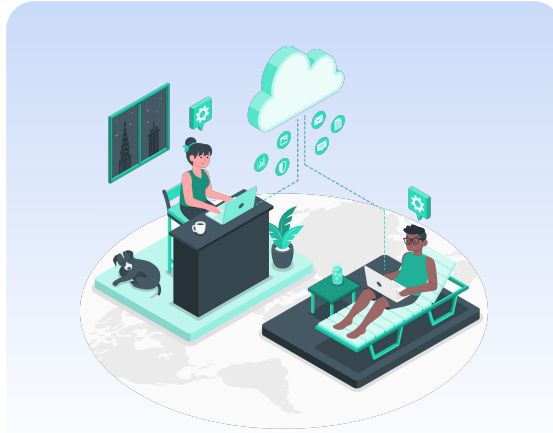
Gaming Applications.

C++

Applications of C++



Big Data Applications.



Cloud-based Applications.



Web Applications.

...many more

Why C++?

Frameworks & Libraries

Desktop Applications



Qt



wxWidgets

Game Development



UNREAL
ENGINE



SFML

Services



Testing



Google
Test - Image



Catch2

Big Data



Thrust

Applications

Companies

Google

Microsoft



Adobe

ORACLE

JPMORGAN CHASE & Co.



Bloomberg

NETFLIX

TESLA

Meta



...more



Powering Advanced Technology: *The F-35 fighter jet, the most **advanced stealth combat aircraft**, relies heavily on C++ for its avionics software.*



Trusted by NASA: *C++ is extensively used in NASA's flight software, **proving its efficiency and reliability** in critical aerospace applications.*



Backbone of Databases: ***MySQL**, one of the world's most widely used database solutions, is written in C++. Many web technologies have C++ at their core.*



TensorFlow

Secret Behind AI/ML: *While AI is often associated with Python, leading **AI/ML libraries** like **TensorFlow, Caffe, and CNTK** are written in C++ for performance, with Python acting as a wrapper.*



Foundation of Operating Systems: *Major desktop OSs—**Windows, Linux, MacOS**—are primarily written in **C++**, making it indispensable in system-level programming.*

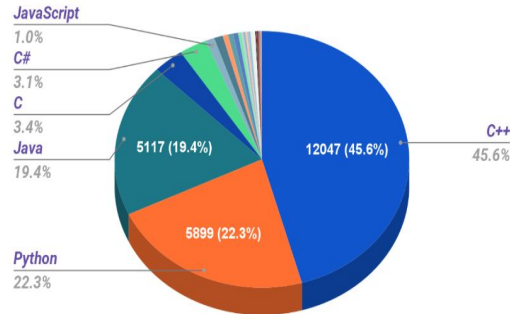
Why C++?

Facts

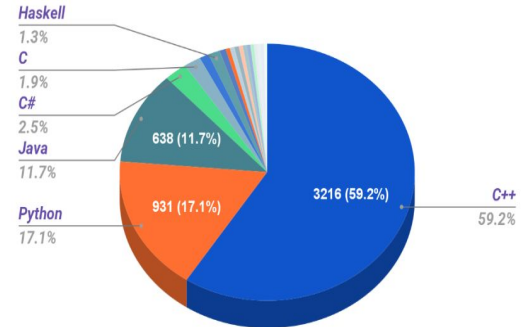


C++ is also highly popular in the **Competitive Programming** Community.

Programming language used (all contestants)



Programming language used (top 20% ranked contestants)



CodeJAM 2017 Qualification Round stats

Compiled Language vs Interpreted Language



v/s



Compiler vs Interpreter

Compiler

C++ uses a **compiler**.

The compiler **translates** the **entire program** into machine code (binary) **before running** it.

Once compiled, the program runs **directly on** the **computer**, making it **very fast**.

Interpreter

Python uses an **interpreter**.

The interpreter **reads** and **runs** the **code line by line** as you execute it.

It **translates** and **runs code** on the **fly**, making it **slower** than **compiled** languages.

Key Difference

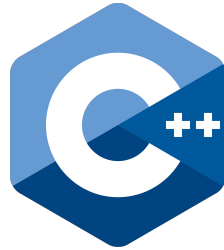
C++

Compile once → Run fast.

Python

*Interpret and run line by line →
Easier to write but slower.*

Boilerplate code



C++

Boilerplate code

C++ 

```
#include <iostream>
```

```
int main() {
```

```
    _____  
    _____  
    _____  
    _____
```

```
    return 0;
```

```
}
```

- *In C++, this library facilitates input and output operations, such as displaying text and reading user input.*

C++

Boilerplate code

C++



```
#include <iostream>
```

```
int main() {
```

```
    _____  
    _____  
    _____  
    _____
```

```
    return 0;
```

```
}
```

- *The main() function serves as the entry point of a C++ program, where execution begins.*
- *It is essential for running the program and plays a key role in controlling its flow.*

C++

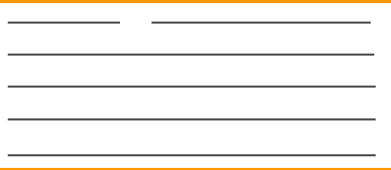
Boilerplate code

C++



```
#include <iostream>
```

```
int main() {
```

A rectangular area with an orange border containing five horizontal lines for code entry. A dashed orange arrow points from this area to the right-hand box.

```
    return 0;
```

```
}
```

Code goes here

C++

Boilerplate code

C++



```
#include <iostream>
```

```
int main() {
```

```
    _____  
    _____  
    _____  
    _____
```

```
    return 0;
```

```
}
```

- *main() returns 0 for success.*
- *Non-zero values indicate errors, enabling execution status checks.*

Syntax: Python v/s C++

Print Statement

C++



```
std::cout << "Output Text" << std::endl;
```

Python



```
print("Output Text")
```

Print Statement

Hello World Program

C++ 

```
#include <iostream>

int main() {
    std::cout << "Hello World!" << std::endl;
    return 0;
}
```

Output 

Hello World!

Print Statement

Hello World Program

C++ 

```
std::cout << "Hello" << std::endl;  
std::cout << "World!" << std::endl;
```

Output 

Hello
World!

*In C++, every statement
ends with a semicolon (;)*

Print Statement

Hello World Program

C++ 

```
#include<iostream>

int main() {
    std::cout << "Hello" << std::endl;
    std::cout << "World" << std::endl;
    return 0;
}
```

Output 

Hello
World

*To avoid writing **std::** multiple times*

Print Statement

Hello World Program

C++ 

```
#include<iostream>
using namespace std;

int main() {
    cout << "Hello" << endl;
    cout << "World" << endl;
    return 0;
}
```

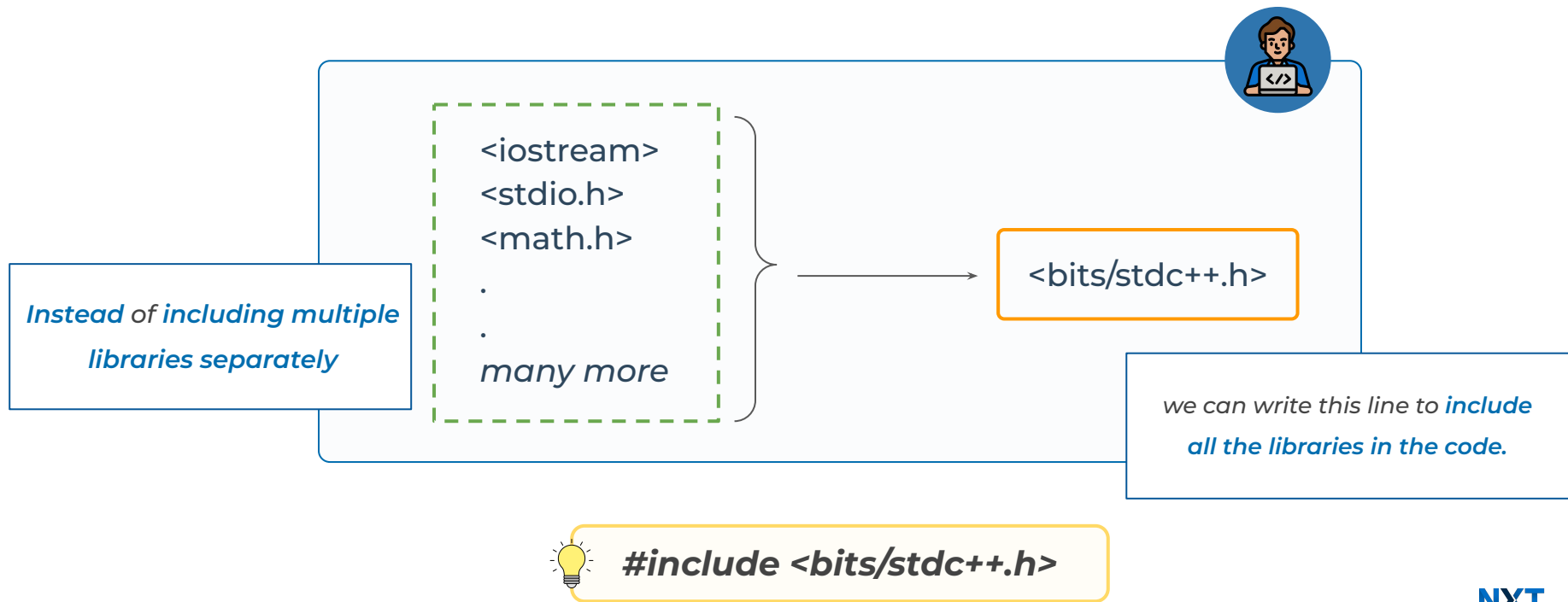
Output 

Hello
World

*To avoid writing **std::** multiple times, add **using namespace std;***

C++

C++ Libraries



C++

Boilerplate code

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {-
```

```
    _____  
    _____  
    _____  
    _____
```

```
return 0;
```

```
}
```

Comments



v/s



Single Line Comment

C++



```
// This is a comment in C++
```

```
cout << "Hello World!" << endl;
```

*C++ uses the **//** symbol to add **single line** comments.*

Python



```
# This is a comment in Python
```

```
print("Hello World!")
```

*Python uses the **#** symbol to add **single line** comments.*

Multi Line Comment

C++



```
/*  
This is a multi line comment  
Printing Hello World!  
*/  
cout << "Hello World!" << endl;
```

C++ uses the `/* */` symbol to add multi line comments.

Python



```
""  
This is a multi line comment  
Printing Hello World!  
""  
print("Hello World!")
```

Python uses the `"""` symbol to add multi line comments.

Variable Declarations



v/s



C++

Variable Declarations

C++



```
bool is_number = true;  
string course = "C++";  
int score = 8;  
float rating = 12.2f;
```

Statically-Typed Language

Need to **declare** the type of variable.

Python



```
is_number = True  
course = "Python"  
score = 8  
rating = 12.2
```

Dynamically-Typed Language

No need to **declare** the type of variable.

Example

C++ 

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string text = "Hello";
    cout << text << endl;
    return 0;
}
```

Output 

Hello

Using Same Variable

C++ 

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int data = 5;
    string data = "Five";
    return 0;
}
```

Error 

main.cpp: In function 'int main()':

main.cpp:6:12: error: conflicting declaration 'std::string data'

```
6 |   string data = "Five";
  |           ^~~~
```

main.cpp:5:9: note: previous declaration as 'int data'

```
5 |   int data = 5;
  |       ^~~~
```


Updating with Different Data Type

C++ 

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int data = 5;
    data = "Five";
    return 0;
}
```

Error 

main.cpp: In function 'int main()':

main.cpp:6:12: error: invalid conversion from 'const char*' to 'int' [-fpermissive]

```
6 |   data = "Five";
  |         ^~~~~~
  |         const char*
```

Quiz Time!



Data Types

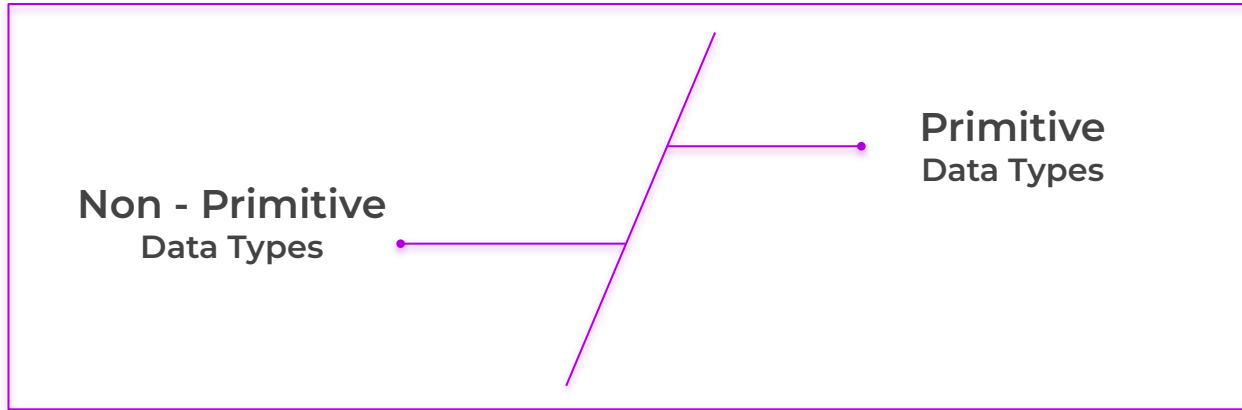


C++

Data Types

C++ supports various data types.

These data types are categorized into:



C++

Primitive Data Types

short

int

long long

float

double

bool

char

C++

Primitive Data Types

<i>Data Type</i>	<i>Description</i>	<i>Bits Acquired</i>	<i>Range of Possible Values</i>	<i>Example</i>
<i>short</i>	<i>Small integer</i>	<i>16 (2 bytes)</i>	<i>-32,768 to 32,767</i>	<i>short s = 32000;</i>
<i>int</i>	<i>Medium integer</i>	<i>32 (4 bytes)</i>	<i>-2,147,483,648 to 2,147,483,647</i>	<i>int i = 123456;</i>
<i>long long</i>	<i>Large integer</i>	<i>64 (8 bytes)</i>	<i>-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807</i>	<i>long long LL = 123456789012345;</i>

C++

Primitive Data Types

<i>Data Type</i>	<i>Description</i>	<i>Bits Acquired</i>	<i>Range of Possible Values</i>	<i>Example</i>
<i>float</i>	<i>Single-precision</i>	<i>32 (4 bytes)</i>	<i>Approx $\pm 3.4 \times 10^{38}$</i>	<i>float f = 12.34f;</i>
<i>double</i>	<i>Double-precision</i>	<i>64 (8 bytes)</i>	<i>Approx $\pm 1.7 \times 10^{308}$</i>	<i>double d = 56.78;</i>
<i>bool</i>	<i>Logical value</i>	<i>8 (1 byte)</i>	<i>true or false</i>	<i>bool flag = true;</i>
<i>char</i>	<i>Single character</i>	<i>8 (1 byte)</i>	<i>0 to 255 (ASCII)</i>	<i>char grade = 'A';</i>

Data Types: Reading Inputs



Data Types

Taking Integer as Input



Taking Integer as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;
```

```
int main() {
```

```
int a;
```

Declaring integer

```
return 0 ;
}
```



Input



Output

Taking Integer as Input

C++



```
#include<bits/stdc++.h>
using namespace std;
```

```
int main() {
```

```
int a;
```

```
cin >> a;
```

Taking input from user

```
return 0 ;
```

```
}
```

Input



5



Output

Taking Integer as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;
```

```
int main() {
int a;
cin >> a;
cout << a;
return 0 ;
}
```

Printing Output

Input 

5

Output 

5

Taking Two Integers as Input

C++



```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
int a, b;
```

Declaring integer

```
return 0 ;
```

```
}
```



Input



Output

Taking Two Integers as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;
```

```
int main() {
    int a, b;
```

```
    cin >> a >> b;
```

*Taking two
integer inputs*

```
    return 0 ;
}
```

Input 

5

10



Output

Taking Two Integers as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;
```

```
int main() {
    int a, b;
    cin >> a >> b;
    cout << a << endl;
    cout << b << endl;
    return 0 ;
}
```

*Printing
a & b*

Input 

5
10

Output 

5
10

Taking Decimal as Input

C++ 

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
int a;
```

We cannot store a decimal value as an integer data type.

```
return 0 ;
```

```
}
```



Input



Output

Taking Decimal as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
int a;
cin >> a;

return 0 ;
}
```

Input 

5.5



Output

Taking Decimal as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int a;
    cin >> a;
    cout << a;
    return 0 ;
}
```

Input 

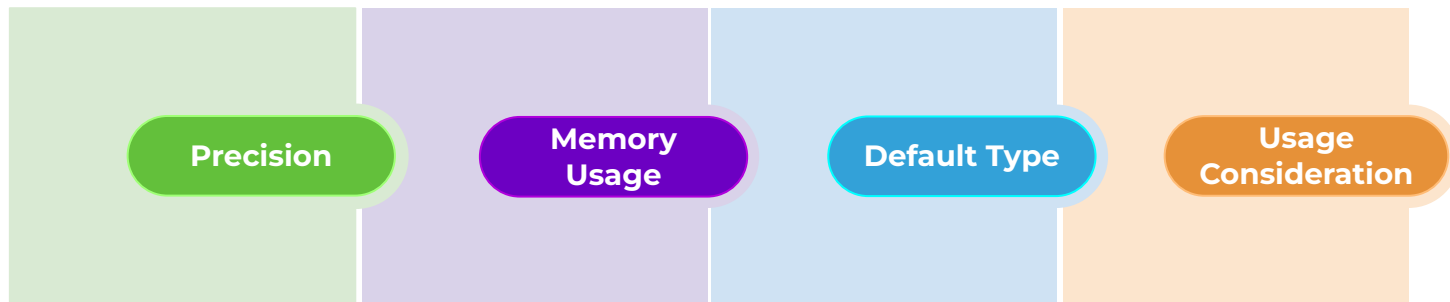
5.5

Output 

5

Taking Decimal as Input

In C++, `float` and `double` are both used to represent real numbers, but they differ in precision and memory usage.



Taking Decimal as Input

In C++, `float` and `double` are both used to represent real numbers, but they differ in precision and memory usage.

Precision

`float`: Approximately 6-7 decimal digits.

`double`: Approximately 15-16 decimal digits.

Memory Usage

`float`: 4 bytes (32 bits).

`double`: 8 bytes (64 bits).

Default Type

In C++, real numbers are treated as `double` by default unless specified as `float`.

Usage Consideration

Use `float` when memory is limited and `double` for higher precision.

float

C++ 

```
#include<bits/stdc++.h>
using namespace std;
```

```
int main() {
    float a;
    cin >> a;
    cout << a;
    return 0;
}
```

Input 

3.14159

 Output

float

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
float a;
cin >> a;
cout << a;
return 0;
}
```

Input 

3.14159

Output 

3.14159

double

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    double b;
    cin >> b;

    return 0;
}
```

Input 

3.141592653589793

 Output

double

C++ 

```
#include<bits/stdc++.h>
using namespace std;
```

```
int main() {
double b;
cin >> b;
cout << b;
return 0;
}
```

Input 

3.141592653589793

Output 

3.14159



In **C++**, the **default precision** is set to **6** significant **digits**.

long long

C++



```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
    long long d;
```

```
    cin >> d;
```

```
    return 0;
```

```
}
```

Input



123465798123456789



Output

long long

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    long long d;
    cin >> d;
    cout << d;
    return 0;
}
```

Input 

123465798123456789

Output 

123465798123456789

Taking Character as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    char a;
    cin >> a;

    return 0;
}
```

Input 

i

 Output

Taking Character as Input

C++



```
#include<bits/stdc++.h>
using namespace std;

int main() {
    char a;
    cin >> a;
    cout << a;
    return 0;
}
```

Input



i

Output



i

C++

Non-Primitive Data Types

*Non-primitive data types are used to **store more complex data structures**, such as arrays and objects.*

string

array

struct

class

... more

Taking Sentence as Input

C++



```
#include<bits/stdc++.h>
using namespace std;

int main() {
    string s1;
    string s2;
    cin >> s1 >> s2;

    return 0;
}
```

Input



Hello World



Output

Taking Sentence as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    string s1;
    string s2;
    cin >> s1 >> s2;
    cout << s1 << endl;
    cout << s2;
    return 0;
}
```

Input 

Hello World

Output 

Hello
World



The *input* will be read into *each string* variable
until a *space* or *newline* is encountered.

Taking Sentence as Input



But what if we had a big
sentence?

Taking Sentence as Input



But what if we had a big sentence?

Use `getline ( );`

Taking Sentence as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;
```

```
int main() {
    string s;
    getline(cin, s);

    return 0;
}
```

Input 

Hello World

 Output

Taking Sentence as Input

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    string s;
    getline(cin, s);
    cout << s;
    return 0;
}
```

Input 

Hello World

Output 

Hello World

Key Differences

String Input

- In **C++**, **cin** stops at **whitespace**, so **getline()** is needed for **multi-word strings**.
- In **Python**, **input()** reads the entire **line** as a **string**.

Key Differences

Input

- C++ uses `cin` for input, while Python uses `input()`.
- Python automatically reads `input` as a string, so we need to convert it to the desired type (e.g., `int()`, `float()`).

Quiz Time!



C++

Key Takeaways (PART 1/4)

- ✓ ***Three Core Areas in Programming***
Focus on Programming Abstractions, Problem-Solving Approach, and Syntax to master any language.
- ✓ ***Compiler vs Interpreter***
Difference between **compiled (C++)** and **interpreted (Python)** languages in terms of **execution speed** and **efficiency**.



C++

Key Takeaways (PART 2/4)

✓ **Boilerplate Code in C++**

Understanding the structure of a C++ program, including `#include <iostream>` and `main()` function.

✓ **Comments in C++**

Using single-line (`//`) and multi-line (`/* */`) comments for documentation.



C++

Key Takeaways (PART 3/4)

✓ ***Variable Declarations***

Understanding **static typing** in **C++** vs **dynamic typing** in **Python**, and how **type declaration** works.

✓ ***Input and Output in C++***

Using **cin** for **input** and **cout** for **output**, along with **endl** for **formatting**.



C++

Key Takeaways (PART 4/4)

- ✓ ***Data Types in C++***

Learning primitive data types (`int`, `float`, `double`, `char`, `bool`, etc.) and their memory allocation

- ✓ ***Practice problems.***





THANK YOU





ALL THE BEST



Practice Questions



Practice questions

Calculate the Area of a Circle

*You're given the radius of a circle,
and your task is to calculate the
area of a circle.*



Calculate the Area of a Circle

Code

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    double radius;
    cin >> radius;
    double area = 3.14 * radius * radius;
    cout << "Area: " << area << endl;
    return 0;
}
```

Input 

5

Output 

Area: 78.5

Python 

```
radius = float(input())
area = 3.14 * radius * radius
print("Area: ", area)
```

Input 

5

Output 

Area: 78.5

Simple Interest Calculation

You're given,

- **Principal amount (P)**,
- **Time (T)** in years,
- Rate of **Interest (R)** per annum and

your task is to **calculate Simple Interest**
using the formula:

$$\text{Simple Interest} = \frac{P \times T \times R}{100}$$

Simple Interest Calculation

Code

C++ 

```
#include<bits/stdc++.h>
using namespace std;
int main() {
    double p, t, r;
    cin >> p >> t >> r;
    double simpleInterest = (p * t * r) / 100;
    cout << "Simple Interest: " << simpleInterest << endl;
    return 0;
}
```

Input 

200 5 2.5

Output 

Simple Interest: 25